RGUKT RK VALLEY

Department of Computer Science and Engineering

SITE RELIABILITY ENGINEERING (SRE) DOCUMENT

E-Commerce Website

Student Name : Raghavendra

ID Number : R220348

Department : CSE

Table of Contents

1. Introduction
2. Project & System Overview
3. System Architecture
4. Reliability Goals
5. Service Level Indicators (SLIs) & Objectives (SLOs)
6. Monitoring & Observability
7. Alerting Strategy
8. Incident Management
9. Deployment & Rollback
1. 10. Scalability & Performance
2. 11. Database Reliability
3. 12. Disaster Recovery
4. 13. Security & Reliability
5. 14. Dependency Management
6. 15. Backup & Data Recovery
7. 16. Capacity Planning
8. 17. Error Budget
9. 18. Reliability Testing
10. 19. Maintenance & Operations
11. 20. SRE Responsibilities
12. 21. Reliability Checklist
13. 22. Conclusion

---

1. Introduction

1.1 Purpose

This document defines the Site Reliability Engineering (SRE) practices, reliability requirements, monitoring strategies, incident procedures, and operational standards for the E-Commerce Website.

The primary goal is to ensure that the application remains:

- Available
- Reliable
- Secure
- Scalable
- Performant
- Recoverable
- Observable

The SRE approach combines software engineering and system administration practices to maintain reliable production services.

1.2 Scope

This document covers the reliability of:

- Customer-facing web application
- Product catalog
- Search
- Authentication
- Shopping cart
- Wishlist
- Checkout
- Order management
- Payment processing
- Inventory management
- Notifications
- Backend APIs
- Database
- Caching
- External services
- Deployment infrastructure

---

2. Project & System Overview

2.1 Project Description

The E-Commerce Website is an online shopping platform where customers can browse products, search for products, add products to their cart, place orders, make payments, and track their orders.

The platform consists of a frontend application, backend APIs, database, caching layer, and external service integrations.

2.2 Major Features

Customer Features

- User registration
- User login/logout
- Product browsing
- Product search
- Product filtering
- Product sorting
- Product details
- Shopping cart
- Wishlist
- Checkout
- Online payment
- Order placement
- Order tracking
- Order history
- Profile management

Admin Features

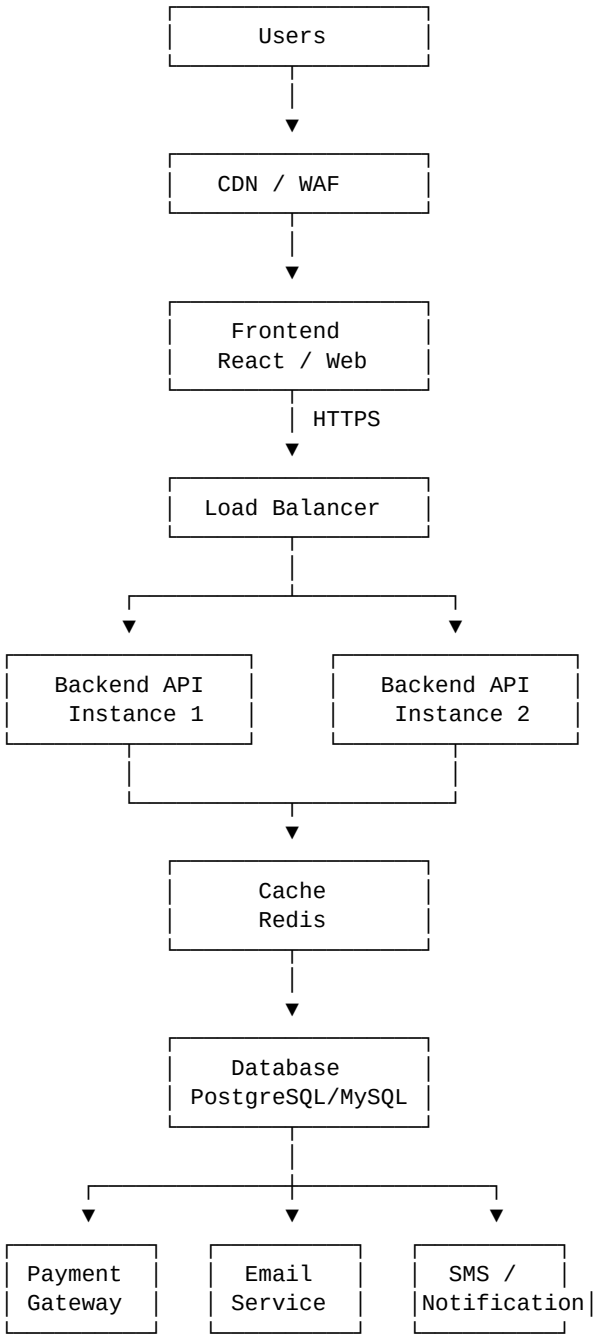
- Product management
- Category management
- Inventory management

- Order management
- User management
- Sales monitoring
- Dashboard and reports

---

3. System Architecture

3.1 High-Level Architecture



3.2 Main Components

ComponentResponsibility	

Frontend	User interface
CDN	Static content delivery
WAF	Web security
Load Balancer	Traffic distribution
Backend API	Business logic
Cache	Fast data access
Database	Persistent data
Payment Gateway	Payment processing
Notification Service	Email/SMS notifications
Monitoring System	System visibility
Logging System	Centralized logs

---

4. Reliability Goals

The primary reliability goals are:

Availability

The website should remain accessible to users with minimal downtime.

Performance

Pages and APIs should respond within defined latency targets.

Scalability

The system should handle increased traffic during:

- Sales
- Festivals
- Product launches
- Promotional campaigns
- Flash sales

Data Integrity

Orders, payments, customer information, and inventory data must remain consistent.

Recoverability

The system should recover quickly from infrastructure or application failures.

Security

Customer and payment-related information must be protected.

---

5. SLI & SLO

5.1 Service Level Indicators

SLIs measure the actual reliability and performance of the system.

Availability SLI

```
Availability =
Successful Requests / Total Requests × 100
```

Latency SLI

Measures the percentage of requests completed within the defined response-time threshold.

Error Rate SLI

```
Error Rate =
Failed Requests / Total Requests × 100
```

Checkout Success Rate

```
Checkout Success Rate =
Successful Checkouts / Checkout Attempts × 100
```

Payment Success Rate

```
Payment Success Rate =
Successful Payments / Payment Attempts × 100
```

5.2 Service Level Objectives

ServiceSLO	
Website Availability	≥ 99.9%
API Availability	≥ 99.9%
Product API Latency	< 500 ms
Checkout API Latency	< 1 sec
Error Rate	< 1%
Payment Success Rate	≥ 99%
Checkout Success Rate	≥ 98%
Database Availability	≥ 99.95%

Availability Target

A 99.9% monthly availability target allows approximately 43.2 minutes of downtime per 30-day month.

---

6. Monitoring & Observability

The system should provide visibility into:

- Infrastructure
- Application
- Database
- Network
- APIs
- User experience
- External dependencies

6.1 Metrics

Infrastructure Metrics

Monitor:

- CPU utilization
- Memory utilization
- Disk usage
- Network traffic
- Instance health

- Container health

Application Metrics

Monitor:

- Request rate
- Response latency
- HTTP status codes
- Error rate
- Active users
- API throughput

Database Metrics

Monitor:

- CPU
- Memory
- Connections
- Query latency
- Slow queries
- Lock contention
- Replication status
- Storage utilization

Business Metrics

Monitor:

- Orders per minute
- Successful checkouts
- Payment failures
- Cart abandonment
- Product searches
- Inventory changes

---

7. Logging

All production services should generate structured logs.

7.1 Log Levels

```
DEBUG
INFO
WARN
ERROR
FATAL
```

Production systems should primarily use:

```
INFO
WARN
ERROR
```

7.2 Important Log Information

Logs should include:

- Timestamp
- Service name

- Request ID
- User/session identifier where appropriate
- HTTP method
- Endpoint
- Response status
- Response time
- Error information

Sensitive information such as passwords, authentication tokens, and payment credentials must never be logged.

---

8. Distributed Tracing

Distributed tracing should be used to understand requests flowing through multiple services.

Example:

```
User
 ↓
Frontend
 ↓
API Gateway
 ↓
Order Service
 ↓
Inventory Service
 ↓
Payment Service
 ↓
Database
```

Each request should have a trace/request identifier that allows engineers to follow the complete request path.

---

9. Alerting Strategy

Alerts should be based on meaningful reliability problems rather than every small metric fluctuation.

9.1 Critical Alerts

Trigger immediately for:

- Website completely unavailable
- API outage
- Database unavailable
- Payment system outage
- Extremely high error rate
- Data corruption
- Security incident

9.2 Warning Alerts

Trigger when:

- CPU remains high
- Memory usage is high
- Disk usage approaches capacity

- API latency increases
- Error rate starts increasing
- Database connections approach the limit

9.3 Example Alert Thresholds

Metric	**Warning**	**Critical**
CPU	> 70%	> 90%
Memory	> 75%	> 90%
Disk	> 75%	> 90%
Error Rate	> 2%	> 5%
API Latency	> 1 sec	> 3 sec
Database Connections	> 70%	> 90%

Thresholds should be tuned using real production behavior.

---

10. Incident Management

10.1 Incident Severity

SEV-1 — Critical

Complete service outage or major business impact.

Examples:

- Website unavailable
- Checkout completely unavailable
- Major payment failure
- Database failure

SEV-2 — High

Major feature degraded but the platform remains partially usable.

Examples:

- Product search unavailable
- Significant API failures
- Slow checkout

SEV-3 — Medium

Limited functionality affected.

Examples:

- Wishlist unavailable
- Notification delays
- Admin dashboard issues

SEV-4 — Low

Minor issues with minimal user impact.

Examples:

- UI defects
- Minor reporting issues

---

11. Incident Response Process

```
Detect
  ↓
Alert
  ↓
Acknowledge
  ↓
Investigate
  ↓
Mitigate
  ↓
Recover
  ↓
Verify
  ↓
Post-Incident Review
```

11.1 Incident Response Principles

During an incident:

1. Identify the impact.
2. Assign an incident owner.
3. Communicate clearly.
4. Focus on mitigation first.
5. Avoid risky changes.
6. Restore service quickly.
7. Investigate the root cause afterward.

---

12. Deployment Strategy

Deployments should be automated through CI/CD.

```
Developer
   ↓
Git Repository
   ↓
CI Pipeline
   ↓
Build
   ↓
Unit Tests
   ↓
Integration Tests
   ↓
Security Checks
   ↓
Deployment
   ↓
Health Checks
   ↓
Production
```

12.1 Deployment Requirements

Before production deployment:

- Code review completed
- Automated tests passed
- Security checks passed
- Database migrations verified
- Health checks successful
- Rollback plan available

---

13. Deployment Strategies

Rolling Deployment

Gradually replace old application instances with new ones.

Advantages:

- Minimal downtime
- Resource efficient
- Simple to operate

Blue-Green Deployment

Maintain two environments:

```
Blue  → Current Production
Green → New Version
```

Traffic can be switched to Green after validation.

Canary Deployment

Release the new version to a small percentage of users first.

```
95% → Old Version
5%  → New Version
```

If the new version is healthy, gradually increase traffic.

---

14. Rollback Strategy

A deployment should be rolled back when:

- Error rate increases significantly
- Latency increases
- Critical functionality fails
- Database errors occur
- Payment processing breaks
- Unexpected behavior is detected

Rollback process:

```
Detect Problem
      ↓
Stop Deployment
      ↓
Assess Impact
      ↓
Rollback Version
      ↓
Verify Health
      ↓
```

```
Monitor
      ↓
Post-Incident Review
```

Database migrations should use backward-compatible strategies whenever possible.

---

15. Scalability & Performance

The system should support both vertical and horizontal scaling.

15.1 Horizontal Scaling

Add more application instances:

```
Traffic
   ↓
Load Balancer
 ↓   ↓   ↓
API API API
```

15.2 Vertical Scaling

Increase:

- CPU
- RAM
- Storage

for existing infrastructure.

Horizontal scaling should generally be preferred for stateless application services.

---

16. Caching Strategy

Caching should be used for frequently accessed data.

Suitable candidates:

- Product information
- Categories
- Popular products
- Search results
- Configuration data

Example:

```
Request
  ↓
Cache
  ↓
Data Available?
 / \
Yes No
 |   |
Return Database
      ↓
    Cache
      ↓
    Return
```

Cache invalidation must be carefully handled to avoid stale product or inventory information.

---

17. Database Reliability

The database is a critical component of the e-commerce platform.

17.1 Reliability Requirements

- Automated backups
- High availability
- Monitoring
- Connection pooling
- Query optimization
- Replication where required
- Storage monitoring
- Disaster recovery

17.2 Database Performance

Monitor:

- Slow queries
- Index usage
- Connection count
- Lock waits
- Query execution time
- CPU utilization
- Storage utilization

17.3 Data Integrity

Transactions should be used for critical operations such as:

```
Order Creation
+
Inventory Update
+
Payment Status Update
```

The system must prevent:

- Duplicate orders
- Negative inventory
- Duplicate payments
- Lost transactions

Idempotency should be implemented for payment and order operations where appropriate.

---

18. Disaster Recovery

Disaster Recovery (DR) ensures that the system can recover after major failures.

Possible disasters include:

- Database failure
- Infrastructure failure
- Region outage
- Data corruption
- Security attack
- Accidental deletion

18.1 Recovery Objectives

RTO — Recovery Time Objective

Maximum acceptable time to restore service.

Target: ≤ 1 hour

RPO — Recovery Point Objective

Maximum acceptable amount of data loss.

Target: ≤ 15 minutes

These targets should be validated against actual infrastructure capabilities.

---

19. Backup Strategy

Backups should include:

- Database
- Application configuration
- Important files
- Infrastructure configuration
- Critical business data

19.1 Backup Types

Full Backup

Complete copy of the database.

Incremental Backup

Only data changed since the previous backup.

Point-in-Time Recovery

Restore the database to a specific point in time using backups and transaction/WAL logs where supported.

19.2 Backup Requirements

Backups should be:

- Automated
- Encrypted
- Versioned
- Stored separately from production
- Regularly tested

A backup that has never been restored successfully should not be considered a reliable recovery mechanism.

---

20. Security & Reliability

Security is an important part of system reliability.

20.1 Security Practices

- HTTPS everywhere

- Secure authentication
- Strong password hashing
- Role-based authorization
- Input validation
- Rate limiting
- WAF protection
- Secure secrets management
- Dependency scanning
- Regular vulnerability scanning

20.2 Sensitive Data

Never store or log:

- Plain-text passwords
- Authentication tokens
- Payment card details
- API secrets
- Private keys

Secrets should be stored in a dedicated secrets-management system.

---

21. Dependency Management

The e-commerce platform may depend on:

- Payment gateways
- Email providers
- SMS providers
- Cloud services
- CDN
- Authentication providers
- Shipping APIs

21.1 Dependency Failure Strategy

External dependency failures should not unnecessarily bring down the entire website.

For example:

```
Payment Service Failure
        ↓
Checkout detects failure
        ↓
User receives clear message
        ↓
Order state remains consistent
        ↓
Payment can be retried
```

Use:

- Timeouts
- Retries
- Exponential backoff
- Circuit breakers
- Rate limits
- Fallbacks

- Idempotency

Retries should be carefully controlled to avoid creating additional load during an outage.

---

22. Rate Limiting

Rate limiting protects services from excessive traffic and abuse.

Example:

```
Login API
→ 5 requests / minute / IP

Search API
→ 60 requests / minute / user

Public API
→ Defined requests / minute / client
```

Limits should be adjusted according to actual traffic patterns.

---

23. Health Checks

Services should expose health endpoints.

Liveness Check

Determines whether the application process is running.

```
GET /health/live
```

Readiness Check

Determines whether the application is ready to receive traffic.

```
GET /health/ready
```

Readiness should verify required dependencies where appropriate.

---

24. Capacity Planning

Capacity should be monitored continuously.

Important resources:

- CPU
- Memory
- Storage
- Database connections
- Network bandwidth
- API throughput
- Cache capacity

Capacity planning should consider expected traffic spikes during:

- Festival sales
- Black Friday-style events
- Product launches
- Promotional campaigns
- Flash sales

Load testing should be performed before major campaigns.

---

25. Error Budget

An error budget represents the amount of unreliability allowed by the SLO.

For a 99.9% availability SLO:

```
Allowed failure = 0.1%
```

For a 30-day month:

```
Total time ≈ 43.2 minutes
```

If the error budget is being consumed too quickly:

- Reduce risky deployments
- Prioritize reliability work
- Investigate recurring incidents
- Improve testing
- Improve infrastructure

If the system is well within its error budget, teams can safely take on more controlled changes and feature releases.

---

26. Reliability Testing

Reliability should be tested continuously.

26.1 Load Testing

Test system behavior under expected traffic.

26.2 Stress Testing

Push the system beyond expected capacity to determine failure behavior.

26.3 Failure Testing

Test:

- API failures
- Database failures
- Cache failures
- Network failures
- Dependency failures

26.4 Disaster Recovery Testing

Regularly verify:

- Backups
- Database restoration
- Recovery procedures
- Failover
- RTO
- RPO

---

27. Maintenance & Operations

Regular maintenance activities include:

Daily

- Check critical alerts
- Review error rates
- Check infrastructure health
- Review important incidents

Weekly

- Review SLO performance
- Review logs
- Check resource utilization
- Review failed deployments

Monthly

- Review error budget
- Test backups
- Review incidents
- Review capacity
- Review dependencies
- Perform security updates

---

28. SRE Responsibilities

Development Team

- Write reliable code
- Add automated tests
- Implement proper error handling
- Add application metrics
- Fix reliability issues

DevOps / SRE Team

- Maintain infrastructure
- Monitor services
- Manage deployments
- Respond to incidents
- Improve scalability
- Maintain observability
- Maintain disaster recovery

Database Team

- Database performance
- Backup management
- Replication
- Recovery testing
- Data integrity

Security Team

- Vulnerability management
- Security monitoring
- Access control
- Security incident response

---

29. Reliability Dashboard

The production dashboard should provide a single view of system health.

Recommended dashboard sections:

```
┌─────────────────────────────────────────────┐
│              SYSTEM HEALTH                  │
├─────────────────────────────────────────────┤
│ Availability     99.95%     ✓               │
│ Error Rate        0.2%       ✓              │
│ API Latency       320ms      ✓              │
│ CPU               54%        ✓              │
│ Memory            61%        ✓              │
│ Database          Healthy    ✓              │
│ Cache             Healthy    ✓              │
│ Payments          Healthy    ✓              │
├─────────────────────────────────────────────┤
│ Active Incidents: 0                         │
│ Error Budget Remaining: 82%                 │
└─────────────────────────────────────────────┘
```

---

30. Reliability Checklist

Application

- Error handling implemented
- Input validation implemented
- Health checks implemented
- Timeouts configured
- Idempotency implemented for critical operations
- Logging implemented
- Metrics implemented

Infrastructure

- Load balancing configured
- Auto-scaling configured
- Resource monitoring configured
- CDN configured where appropriate
- WAF/security controls configured

Database

- Automated backups enabled
- Backup restoration tested
- Query monitoring enabled
- Connection limits monitored

- Storage monitored
- Recovery procedure documented

Deployment

- CI/CD configured
- Automated tests enabled
- Deployment health checks enabled
- Rollback procedure documented
- Database migrations tested

Monitoring

- Metrics available
- Centralized logging available
- Alerts configured
- Dashboards available
- SLO tracking enabled

Disaster Recovery

- RTO defined
- RPO defined
- Backup strategy documented
- Recovery procedure tested
- Failover procedure documented

---

31. Key Reliability Targets

CategoryTarget	
Website Availability	≥ 99.9%
API Availability	≥ 99.9%
Database Availability	≥ 99.95%
API Latency	< 500 ms
Checkout Latency	< 1 sec
Error Rate	< 1%
Payment Success	≥ 99%
RTO	≤ 1 hour
RPO	≤ 15 minutes
Critical Incident Response	Immediate
Backup	Automated
Rollback	Tested

---

32. Conclusion

The SRE strategy for the E-Commerce Website focuses on maintaining high availability, performance, scalability, security, and data integrity.

The system should continuously measure reliability through SLIs and SLOs, monitor application and infrastructure health, respond quickly to incidents, use safe deployment strategies, maintain reliable backups, and regularly test disaster recovery procedures.

Reliability is treated as a continuous engineering process rather than a one-time configuration.

The overall objective is:

```
Reliable System
      ↓
Better Performance
      ↓
Fewer Incidents
      ↓
Faster Recovery
      ↓
Better User Experience
      ↓
Higher Customer Trust
```

End of SRE Document